

LETTER

Formal and Empirical Analysis of an Authorization Decision Inconsistency in SAGA

TODO AUTHOR[†], ??

SUMMARY We analyze an order-dependent authorization inconsistency in SAGA: a later wildcard allow can replace a more-specific deny because the matcher fails to retain the best pattern. In controlled executions, the implementation decision propagates through production consumer paths to token issuance and storage. Under the fixed divergence, ProVerif reports ChatAccept reachable and the prior-TokenIssue correspondence true.

key words: authorization, access control, formal verification, AI agents, ProVerif

1. Introduction

SAGA [1] uses contact policies and access-control tokens to authorize communication between registered agents. Authentication establishes identity but does not imply permission under the receiver's contact policy.

We identify an order-dependent inconsistency in the production matcher. Under the documented most-specific semantics, the specific deny should be selected over the wildcard allow. Instead, the matcher returns the later wildcard allowance; reversing the rule order returns denial.

In controlled executions, the implementation decision propagates through production consumer paths to token issuance and storage. A separate symbolic model analyzes acceptance under the supplied divergence.

Our contributions are (1) identifying and explaining the matcher inconsistency, and (2) characterizing its consequences through controlled production-consumer propagation and conditional symbolic analysis.

(1) Related work.

Prior work covers distributed authorization semantics [2], [3], authorization credentials [4], and policy change-impact analysis [5]. ProVerif and Tamarin support symbolic protocol analysis [6], [7]; SAGA uses ProVerif [1]. We analyze the concrete rule-selection deviation and downstream consumer effects, complementing SAGA's protocol-level guarantees under its assumptions.

2. Authorization Decision Inconsistency

(1) Workflow and policy semantics.

An initiator requests contact through SAGA's Provider. The receiver's ordered policy maps identity patterns to integer budgets. Provider and receiver independently call the production matcher on the initiator. The Provider gates one-time

Table 1 Specification and production-matcher results under two rule orders.

Rule order	Spec. budget	Matcher budget
Specific deny, then wildcard allow	-1	10
Wildcard allow, then specific deny	-1	-1

key (OTK) release; the receiver rechecks its policy before token issuance.

For a fixed policy and valid identity, SAGA v2, Section IV-D.1 [1], specifies that the most-specific matching rule is selected; the implementation scores patterns with `aid_specificity()`. A positive budget permits the request; a negative budget denies it. We do not consider equal-specificity conflicts, policy changes during evaluation, or zero budgets.

(2) Root cause and observation.

The tested production matcher is `match()` in `saga/common/contact_policy.py`, unchanged from upstream SAGA revision 7372111, also the matcher-test baseline in our study copy.[†] Its `best_pattern` starts at `None`, whose specificity is `-1`. A matching rule with greater specificity updates the budget but never `best_pattern`. Thus later matching rules still compare against `-1`, allowing even a less-specific wildcard to overwrite the budget. In the recorded case, `aid_specificity()` scores `alice@example.com:*` as 70 and `*` as 0; their budgets are `-1` and 10, respectively.

An independent finite specification evaluator selects the specific denial for both orders in Table 1. When specific deny precedes wildcard allow, the specification denies with budget `-1`, but the production matcher returns 10 and permits. The reverse order returns the specified denial, establishing order dependence. These calls establish only a local decision inconsistency, not a deployed exploit, message acceptance, or whole-system failure.

3. Formal Consequence Analysis

(1) Decision input.

The symbolic model fixes the recorded divergence: the specification denies and the observed implementation permits.

[†]TODO: affiliation and postal address.
DOI: 10.1587/transfun.E0.A.1

[†]Upstream: github.com/gsiros/saga; study copy and artifacts: github.com/g1020268262-a11y/saga.

ProVerif assumes these facts; it does not run Python or the production matcher, discover the inconsistency, or establish implementation refinement.

(2) Model assumptions.

The valid registered initiator follows the prescribed symbolic peer process and cannot alter the receiver's fixed policy; reachability is not an arbitrary-adversary reachability claim. The Provider and Certificate Authority (CA) are trusted, cryptographic checks are effective, and private keys remain uncompromised; a private symbolic channel abstracts authenticated, confidential transport.

In the symbolic receiver, $\text{TokenIssue}(R, I, t)$ occurs after ActiveToken stores receiver R , initiator I , encrypted token t , and I 's public access-control key, before t is sent. For private-channel message m , $\text{ChatAccept}(R, I, t, m)$ occurs only after checking the fixed identity and an entry matching those four fields.

(3) Queries and results.

The archived ProVerif 2.05 run (proofs/evidence, 22 September 2026) reports ChatAccept reachable under the fixed divergence. Separately, it reports this non-injective correspondence as true:

$$\text{ChatAccept}(R, I, t, m) \implies \text{TokenIssue}(R, I, t)$$

It requires prior TokenIssue for the same receiver R , initiator I , and encrypted token t used by ChatAccept ; this establishes token-origin consistency, not intended policy permission. Under the fixed divergence, it coexists with specification denial. Issuance appears in the ChatAccept witness; no standalone TokenIssue reachability query exists.

The executions are compared only at authorization decisions and token lifecycle events, with no object-level refinement; only the symbolic model reaches message acceptance.

4. Consumer-Side Propagation Validation

(1) Environment.

At consumer-run revision 8cc5dd7, the relevant matcher, Provider, receiver, and token-cryptography files match upstream revision 7372111. The harness drives the production Provider /access route and receiver handler from matcher evaluation through `active_tokens` storage. Signed registration material, local certificates, and in-memory transport are used with production cryptographic checks active. Wrappers record matcher results without presetting authorization or token state; a boundary stub stops before the original `receive_conversation`.

(2) Cases and controls.

Provider and receiver use identical policies. Divergent D/A places specific deny (−1) before wildcard allow (10); controls are A/A (both rules allow with 10) and D/D (wildcard allow 10 before specific deny −1). Each ran once

Table 2 Divergent-case propagation through production consumer paths; message acceptance was not executed.

Evidence point	Divergent case result
Specification	Deny (−1)
Provider matcher	Allow (10)
Provider /access	HTTP 200; OTK released
Receiver matcher	Allow (10)
Receiver	OTK consumed
Endpoint	Token issued and stored

with fresh state and one OTK. The labels denote specification/observed implementation, not Provider/receiver outcomes. A/A reaches token sending and storage; D/D returns HTTP 403 with no OTK release or state change and never reaches the receiver, so receiver rejection remains untested.

(3) Result and evidence.

Table 2 records both matchers returning 10, OTK release and consumption, and one token generated, sent, and stored in `active_tokens`. The endpoint excludes message acceptance: no application message is sent, and the original `receive_conversation` is not invoked.

5. Scope and Conclusion

Our evidence is limited to one recorded SAGA case and the tested policies; it does not establish correctness for arbitrary policies or any Python–ProVerif refinement. Empirical execution stops before message acceptance; no deployed exploit is established. The inconsistency propagates through controlled production paths to token issuance and storage; symbolic ChatAccept is separately reachable under the fixed divergence.

References

- [1] G. Syros, A. Suri, J. Ginesin, C. Nita-Rotaru, and A. Oprea, “SAGA: A security architecture for governing AI agentic systems,” arXiv preprint arXiv:2504.21034v2, 2025. Version 2, revised 29 August 2025.
- [2] M.Y. Becker and P. Sewell, “Cassandra: Distributed access control policies with tunable expressiveness,” 5th IEEE International Workshop on Policies for Distributed Systems and Networks (POLICY), 2004.
- [3] M.Y. Becker, C. Fournet, and A.D. Gordon, “Design and semantics of a decentralized authorization language,” 20th IEEE Computer Security Foundations Symposium (CSF), pp.3–15, IEEE, 2007.
- [4] A. Birgisson, J.G. Politz, Ú. Erlingsson, A. Taly, M. Vrable, and M. Lentzner, “Macaroons: Cookies with contextual caveats for decentralized authorization in the cloud,” Network and Distributed System Security Symposium (NDSS), Internet Society, 2014.
- [5] K. Fisler, S. Krishnamurthi, L.A. Meyerovich, and M.C. Tschantz, “Verification and change-impact analysis of access-control policies,” 27th International Conference on Software Engineering (ICSE), pp.196–205, 2005.
- [6] B. Blanchet, “Modeling and verifying security protocols with the applied pi calculus and ProVerif,” Foundations and Trends in Privacy and Security, vol.1, no.1–2, pp.1–135, 2016.
- [7] S. Meier, B. Schmidt, C. Cremers, and D. Basin, “The TAMARIN prover for the symbolic analysis of security protocols,” Computer Aided Verification (CAV), Lecture Notes in Computer Science, vol.8044, pp.696–701, Springer, 2013.